SkillUp

Lab: Security Compliance, Performance, and Scalability Analysis with Generative AI

Estimated time: 30 minutes

Learning objectives

After completing this lab, you will be able to:

- Generate security compliance checklists using generative AI
- Identify performance optimization recommendations using generative AI
- Develop scalability plans using generative AI

Introduction

This lab focuses on using generative AI to create security compliance checklists, suggest performance improvements, and outline plans for scaling a system. These are key steps when building or updating systems that must meet regulations such as GDPR or ISO 27001, work efficiently, and handle growth. By working through each step, you will understand how AI can support complex planning and problem-solving tasks.

Note: Make sure you have access to an AI-powered text generation tool before starting.

Exercise 1: Generate a security compliance checklist using AI

In this exercise, you will use generative AI to create a compliance checklist based on sample system requirements. The checklist will help ensure that your system follows important regulations, such as GDPR or ISO 27001. This step is important for anyone building systems that must protect user data or meet legal standards.

Step 1: Generate a compliance checklist from requirements

1. Open your AI-powered text generation tool.
2. Provide the following sample system requirements to the AI tool:

```
The customer management system stores personal information, including names, email addresses, phone numbers, and purchase history. The system allows users to update their profiles, request data delet
```

3. Prompt the AI to generate a compliance checklist based on your understanding.
4. Review the generated compliance checklist from the AI tool.

Hint

Sample prompt:

```
Create a GDPR compliance checklist based on these requirements. For each compliance requirement, indicate whether it is covered by the system and provide a brief explanation.
```

Potential AI output:

- Right to Access (Art. 15) - ✓ Covered: System allows users to download their data.
- Right to Rectification (Art. 16) - ✓ Covered: Users can update their profiles.
- Right to Erasure (Art. 17) - ✓ Covered: Users can request data deletion.
- Data Encryption (Art. 32) - ✓ Covered: Data encrypted in transit and at rest.
- Audit Trails (Art. 30) - ✓ Covered: System logs access and maintains a 12-month audit trail.
- Right to Data Portability (Art. 20) - ✓ Covered: Users can download their data.
- Privacy by Design (Art. 25) - ✓ Partially Covered: Encryption implemented, but needs to verify other privacy measures.
- Data Breach Notification (Art. 33) - ⚠ Not Specified: Requirements don't mention breach notification procedures.
- Consent Management (Art. 7) - ⚠ Not Specified: Requirements don't mention how consent is obtained and tracked.
- Data Processing Agreement (Art. 28) - ⚠ Not Specified: No mention of third-party processor agreements.

Step 2: Identify gaps in compliance coverage

1. Review the compliance checklist generated by the AI tool.
2. Prompt the AI to recommend improvements based on your understanding.
3. Review the AI's recommendations for addressing compliance gaps.

Hint

Sample prompt:

```
Based on this compliance checklist, recommend specific actions to address any gaps or partially covered requirements.
```

Potential AI output:

- Implement a data breach notification system that alerts users within 72 hours of discovering a breach.
- Create a consent management interface that allows users to grant, withdraw, and view their consent preferences.
- Document and track all consent given by users with timestamps and version history.
- Establish data processing agreements with any third-party services that handle personal data.
- Conduct a Data Protection Impact Assessment (DPIA) to verify privacy by design measures.
- Implement automated data retention policies that delete data after specified periods.
- Add user notification features for any changes to privacy policies or data usage.
- Create a process for responding to data subject requests within required timeframes.

Exercise 2: Create performance optimization and scalability plans using AI

In this exercise, you will use generative AI to identify performance improvements and create a scalability plan for a system. You will provide system details to the AI and prompt it to suggest specific optimizations and scaling strategies.

Step 1: Generate performance optimization recommendations

1. Open your AI-powered text generation tool.
2. Provide the following system performance information to the AI:

```
The e-commerce platform currently handles 10,000 concurrent users during peak hours. Average page load time is 3.5 seconds. The database queries take 1.2 seconds on average. The system uses a single-
```

3. Prompt the AI to suggest performance improvements based on your understanding.
4. Review the generated performance recommendations from the AI tool.

Hint

Sample prompt:

```
Analyze this system and suggest specific performance optimizations to reduce load times and improve response times. Prioritize the recommendations by impact.
```

Potential AI output:

- HIGH IMPACT: Implement Redis or Memcached caching layer to cache frequent database queries — Expected improvement: Reduce load time by 40–50%.
- HIGH IMPACT: Add a Content Delivery Network (CDN) for serving static assets and images — Expected improvement: Reduce page load time by 30–40%.
- HIGH IMPACT: Enable image compression and use WebP format for images — Expected improvement: Reduce bandwidth by 60–70%.
- MEDIUM IMPACT: Implement database query optimization and add appropriate indexes — Expected improvement: Reduce query time from 1.2 s to 0.3 s.
- MEDIUM IMPACT: Enable gzip compression for text-based assets (HTML, CSS, JS) — Expected improvement: Reduce payload size by 50–60%.
- MEDIUM IMPACT: Implement lazy loading for images below the fold — Expected improvement: Reduce initial page load by 25%.
- LOW IMPACT: Minify CSS and JavaScript files — Expected improvement: Reduce file size by 15–20%.
- LOW IMPACT: Implement HTTP/2 for multiplexed connections — Expected improvement: Reduce connection overhead by 10–15%.

Step 2: Create a scalability plan

1. Using the same system information from Step 1, add the following scaling goals:

```
The platform needs to scale to support 50,000 concurrent users within 6 months and 100,000 concurrent users within 12 months. The system must maintain 99.9% uptime and keep page load times under 2 se
```

2. Prompt the AI to create a scalability plan based on your understanding.
3. Review the generated scalability plan from the AI tool.

Hint

Sample prompt:

```
Create a detailed scalability plan to achieve these goals. Include infrastructure changes, load balancing strategies, and monitoring recommendations.
```

Potential AI output:

Phase 1 (Months 1–2):

- Migrate to cloud infrastructure with auto-scaling capabilities
- Implement horizontal scaling with a load balancer distributing traffic across 3–5 application servers.
- Set up database replication with read replicas to distribute query load.

Phase 2 (Months 3–4):

- Implement microservices architecture to separate concerns and scale independently
- Deploy containerization using Docker and orchestration with Kubernetes.
- Implement message queue system (RabbitMQ or Kafka) for asynchronous processing.

Phase 3 (Months 5–6):

- Scale to multi-region deployment for geographic distribution
- Implement an advanced caching strategy with multiple cache layers.
- Set up CDN with edge locations for global content delivery.

Monitoring:

- Implement APM tools (New Relic, Datadog) to track performance metrics in real-time.
- Set up automated alerts for CPU usage >80%, memory >85%, response time >2 s.
- Implement distributed tracing to identify bottlenecks across services.

Load Balancing:

- Use Layer 7 load balancing with health checks and automatic failover.
- Implement session persistence using sticky sessions or distributed session storage.

Testing:

- Conduct load testing at 50,000 and 100,000 concurrent users before production rollout.

Step 3: Generate stress-test scenarios

Using the scalability plan from Step 2, prompt the AI to create stress-test scenarios based on your understanding.

Review the generated stress-test scenarios from the AI tool.

Hint

Sample prompt:

```
Based on this scalability plan, create three stress-test scenarios that will validate the system can handle peak loads and recover from failures.
```

Potential AI output:

- Scenario 1 – Peak Load Test: Simulate 100,000 concurrent users accessing the platform simultaneously. Monitor response times, error rates, and server resource utilization. Verify that page load times remain under 2 seconds and the error rate stays below 0.1%. The test should run for 30 minutes to identify any memory leaks or performance degradation over time.
- Scenario 2 – Database Failover Test: While under moderate load (25,000 concurrent users), simulate primary database failure and verify automatic failover to read replica. Measure downtime duration (target: <30 seconds) and data consistency after failover. Validate that no transactions are lost during the failover process.
- Scenario 3 – Cascading Failure Recovery: Simulate multiple simultaneous failures, including one application server crash, CDN degradation, and cache layer failure under high load (75,000 concurrent users). Verify that the load balancer redirects traffic appropriately, cached content falls back to origin servers, and the system maintains at least 80% functionality during the incident. Monitor recovery time to full capacity (target: <5 minutes).

Summary

In this lab, you used generative AI to generate security compliance checklists, identify performance optimization opportunities, create scalability plans, and develop stress-test scenarios. These skills help you apply AI to real-world planning and system analysis tasks.